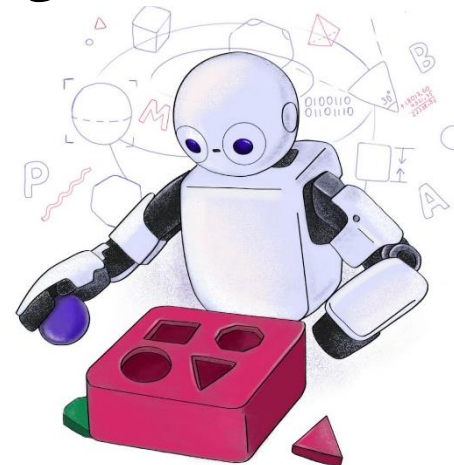


TP557 - Tópicos avançados em IoT e Machine Learning: *Motion Classification*



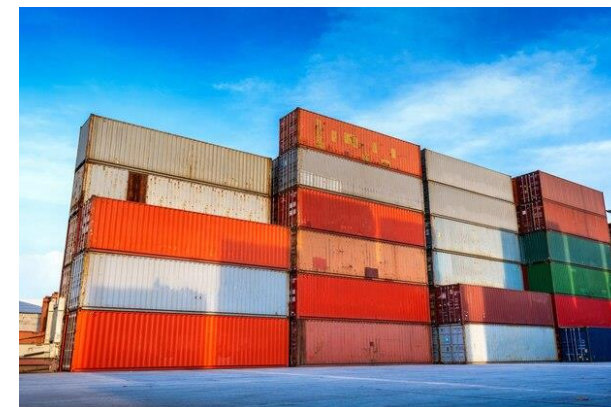
Inatel

Samuel Baraldi Mafra
samuelbmafra@inatel.br

Motion Classification



Rastreamento e
queda de
dispositivos



Controle
logístico



Monitoramento
de animais

Exercícios físicos, movimentar de cadeira de
rodas, interação com máquinas, etc.

Inatel

**CAMINHOS
QUE CONECTAM
COM O FUTURO**

Motion Classification

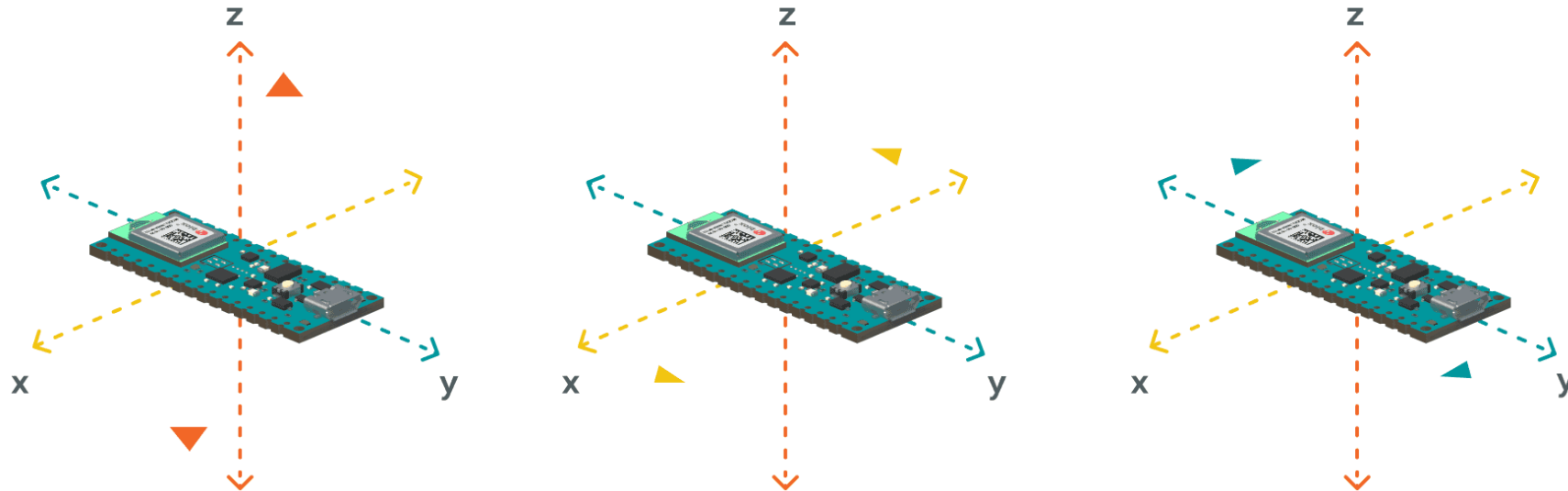
- Movimentação de containers ao longo do ciclo logístico usando um acelerômetro.



Motion Classification

- Movimentação de containers ao longo do ciclo logístico usando um acelerômetro.

Um acelerômetro é um dispositivo usado para medir *forças de aceleração*.
O **LSM9DS1** mede a aceleração em 3 eixos.



Motion Classification

- Container parado no porto:



Movimentação em nenhum eixo.

Motion Classification

- Container sendo erguido:



Movimentação no eixo z.

Motion Classification

- Container transportado no oceano:



- Movimentação em várias direções (x,y,z) devido às correntes marítimas.

Motion Classification

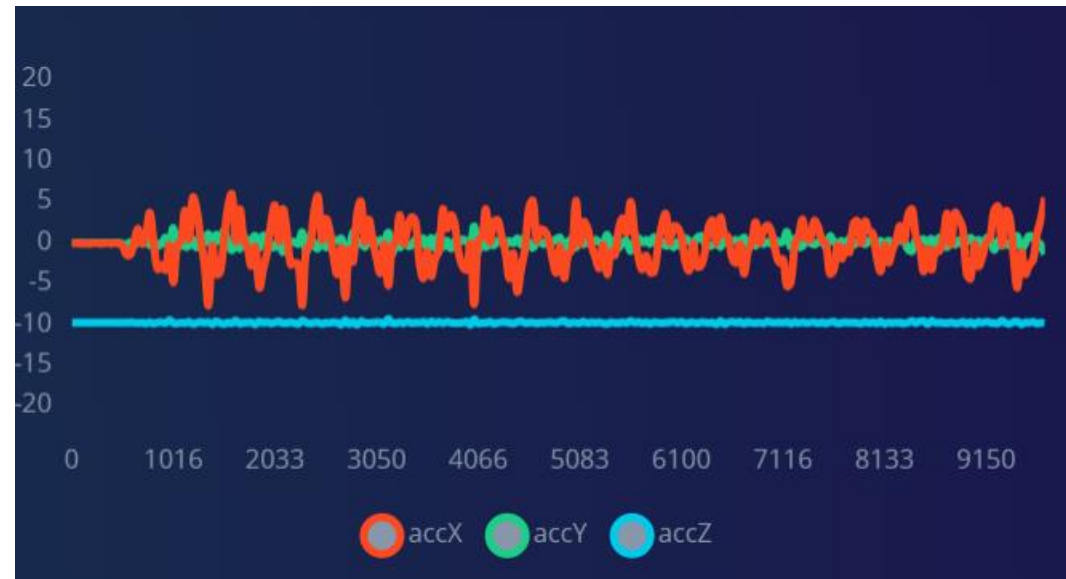
- Container transportado no caminhão:



- Movimentação no eixo x.

Motion Classification

- Exemplo de amostra coletada com acelerômetro:



- Movimentação no eixo x.

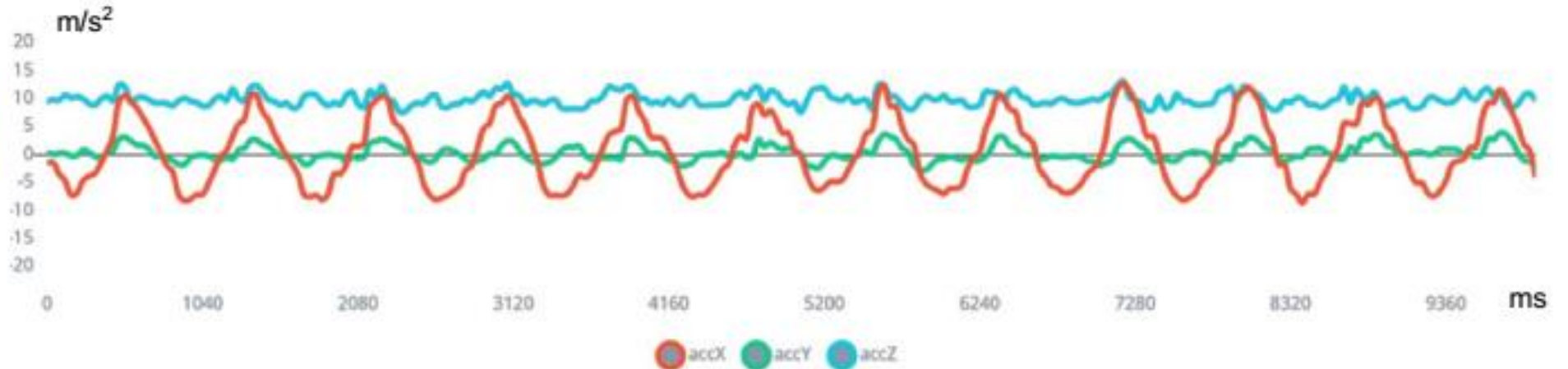
Motion Classification

Aplicação: Classificação de 4 estados do container

- Parado
- Sendo erguido
- No mar
- No caminho

Motion Classification

- Exemplo de 10 segundos de dados coletados por um acelerômetro com frequência de leitura de 62.5 Hz, i.e. 62.5 amostras por segundo.
- Que tipo de dado é este?



Motion Classification

- **Série temporal**: cada amostra tem *correlação* com as *anteriores e posteriores*.
- Assim, por serem séries temporais, vamos *processar janelas que capturem pelo um ou dois ciclos de movimento*, por exemplo, 2 segundos.



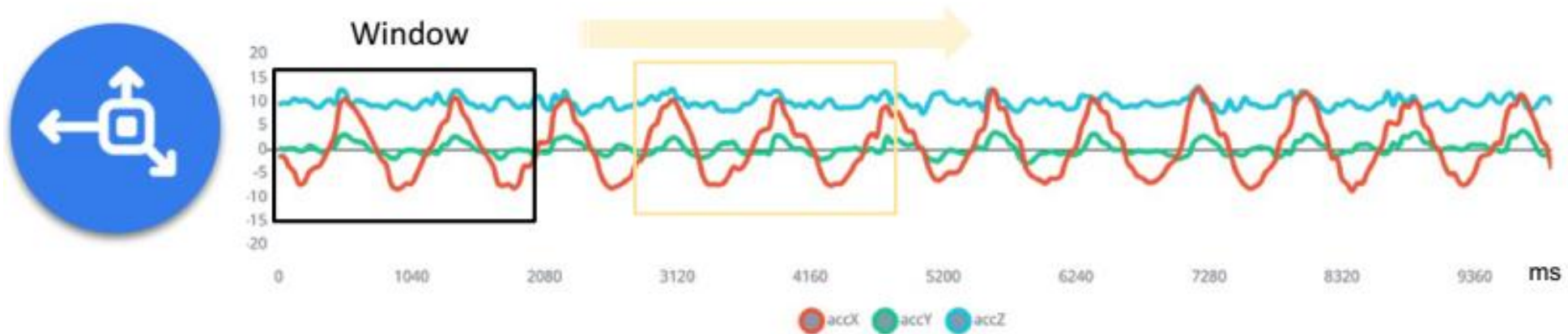
Motion Classification

- Que tipo de rede devemos usar?



Motion Classification

- **Redes com memória, para que elas se lembrem do passado!**
- Redes baseadas em camadas do tipo Long Short-Term Memory (LSTM) e Gated Recurrent Units (GRU), por exemplo.



Motion Classification



Raw Features as a window

- 125** samples for each axis (62.5Hz x 2s)
- 375 total features (125 x 3 axis)

375 atributos de entrada!

* 2 seconds is needed to capture 1 or 2 cycles of movement

** 2 seconds at sample rate of 62.5 Hz -> 125 samples

Motion Classification



Raw Features as a window

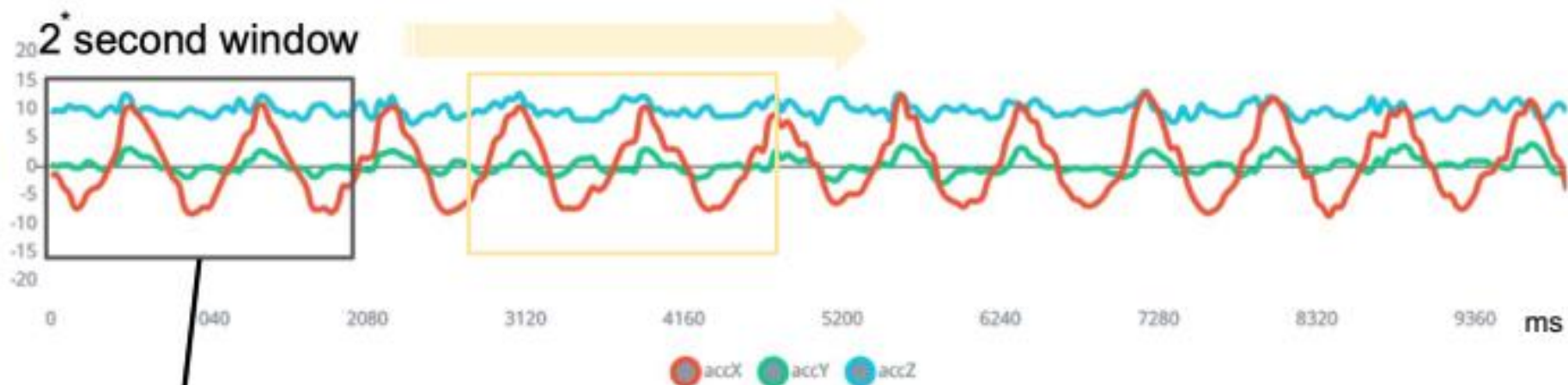
- 125** samples for each axis (62.5Hz x 2s)
- 375 total features (125 x 3 axis)

* 2 seconds is needed to capture 1 or 2 cycles of movement

** 2 seconds at sample rate of 62.5 Hz -> 125 samples

*Precisamos de uma rede complexa (e.g., LSTM) para processar todos os atributos de entrada e **compreender os dados de cada janela.***

Motion Classification



Raw Features as a window

- 125** samples for each axis (62.5Hz x 2s)
- 375 total features (125 x 3 axis)

* 2 seconds is needed to capture 1 or 2 cycles of movement

** 2 seconds at sample rate of 62.5 Hz -> 125 samples

A consequência é que o modelo consome mais memória e pode não operar em tempo real.

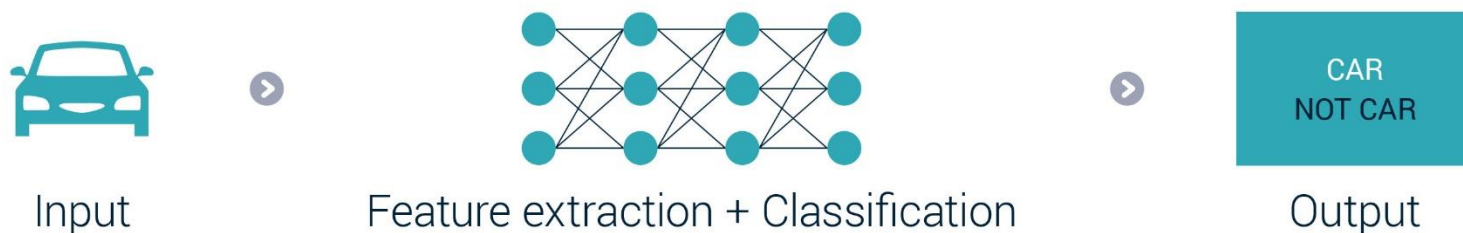
O que podemos fazer?

Motion Classification

Machine Learning



Deep Learning



*Ao invés de termos uma rede muito complexa que faça a **extração de atributos de forma automática**, vamos pré-processar os dados e **extrair alguns atributos manualmente**.*

Motion Classification



375 Raw Features

- Raw Data from sensor



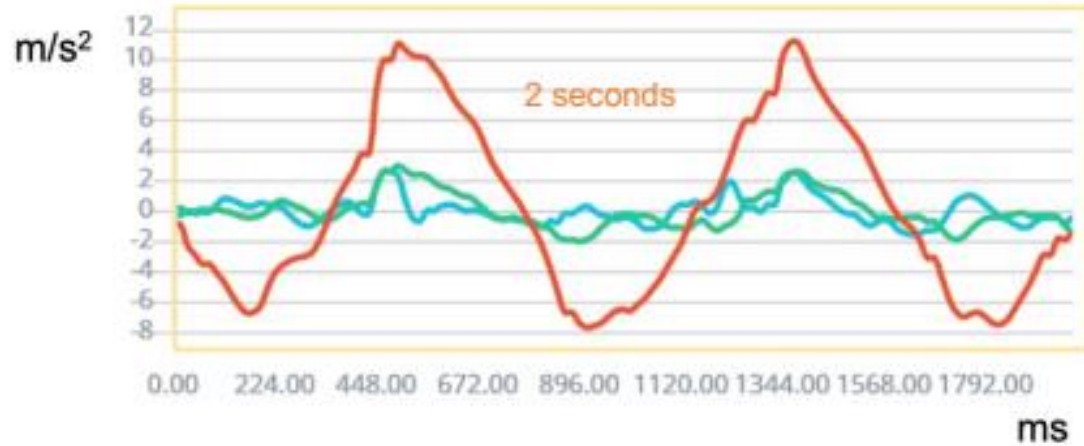
Spectral
features



*Etapa do pré-processamento
usando o bloco “spectral
features*”*

*<https://docs.edgeimpulse.com/docs/edge-impulse-studio/processing-blocks/spectral-features>

Motion Classification



Manual Feature Extraction

RMS (Root Mean Square)

$$x_{\text{RMS}} = \sqrt{\frac{1}{n} (x_1^2 + x_2^2 + \dots + x_n^2)}.$$

Estimativa do desvio padrão

$$\text{Kurt}[X] = E\left[\left(\frac{X - \mu}{\sigma}\right)^4\right] = \frac{E[(X - \mu)^4]}{(E[(X - \mu)^2])^2} = \frac{\mu_4}{\sigma^4}$$

Descreve a frequência com que valores extremos (outliers) ocorrem

$$\gamma_1 := \tilde{\mu}_3 = E\left[\left(\frac{X - \mu}{\sigma}\right)^3\right]$$

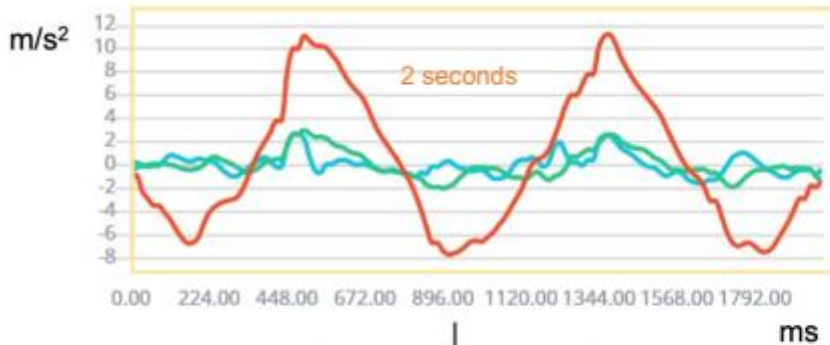
Mede a falta de simetria de uma distribuição

1. Etapa do pré-processamento:

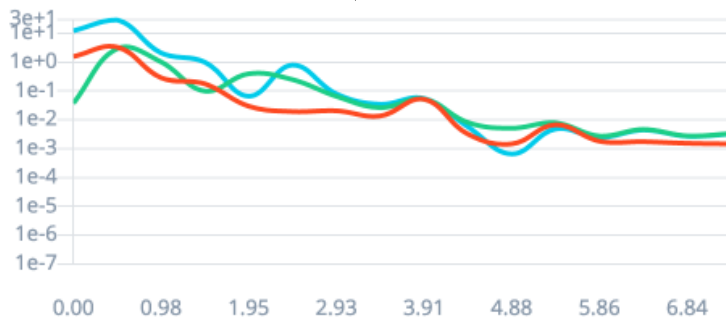
- *Filtra-se os sinais dos 3 eixos,*
- *Calcula-se o valor RMS, Skewness e Kurtosis de cada um dos eixos (x,y e z).*

Total de 9 features

Motion Classification



Feature
extraction:
FFT



Manual Feature Extraction

2. Etapa do pré-processamento:
Calcular a FFT dos sinais do 3 eixos para extrair as frequências dos sinais.

- **Aplica-se FFT à subframes da janela,**
- **Descarta-se a parte espelhada da FFT, os bins acima da frequência de corte do filtro e o bin representando a parte DC (média do sinal),**
- **Aplica log na base 10 a cada bin,**
- **Escolhe-se o bin com maior valor entre dos frames.**

Com uma FFT de 16 pontos e um filtro com corte em 15.625 Hz, temos um total de 12 features

Motion Classification



- 375 Raw Features
- Raw Data from sensor

Manual Feature
Extraction

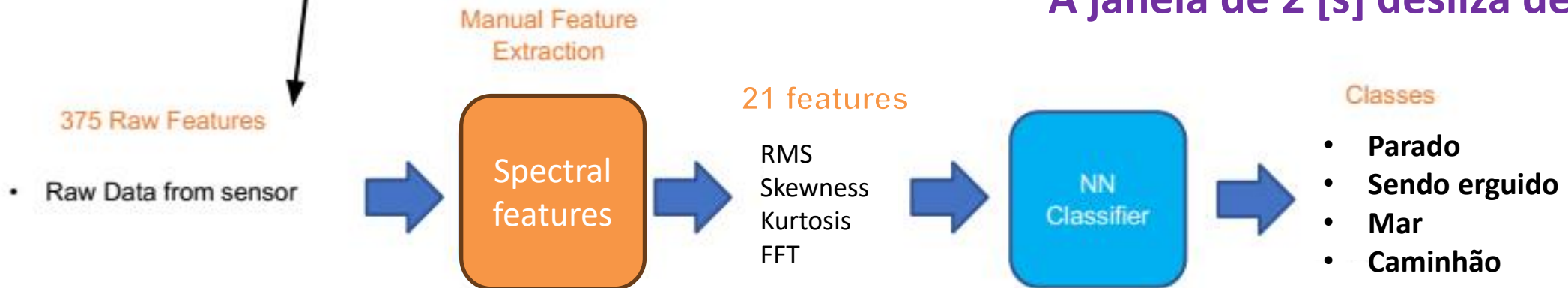
Spectral
features

*Portanto, ao invés de
375 a rede neural terá
21 atributos de entrada.*

Motion Classification



A janela de 2 [s] desliza de 80 [ms].



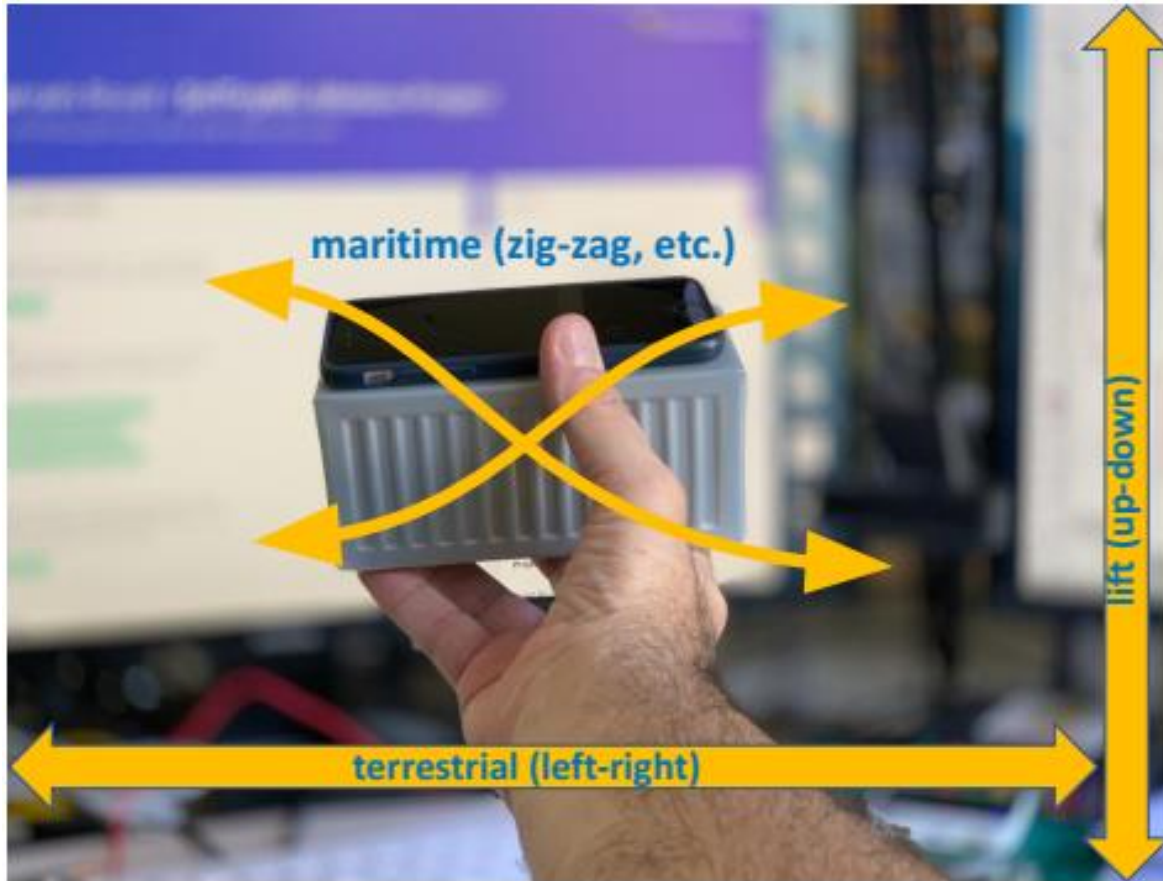
Motion Classification



*Vamos usar o Edge Impulse para criar essa aplicação.
Com ele, vamos desde a coleta de dados, passando pelo
pré-processamento, até a inferência no dispositivo.*

OBS.: O Edge Impulse faz toda a análise espectral (i.e., o pré-processamento) de forma automática.

Motion Classification



Transportation Classes

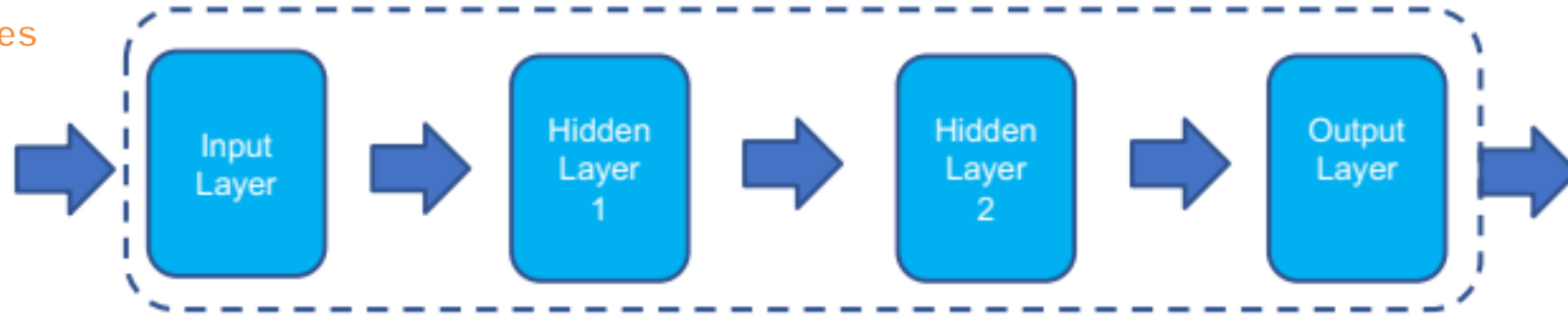
- **lift** (up-down)
- **terrestrial** (left-right)
- **maritime** (zig-zag, etc.)
- **idle**

*Vamos criar a base de dados
no Edge Impulse com o
acelerômetro do celular.*

Motion Classification

21 features

RMS
Skewness
Kurtosis
FFT



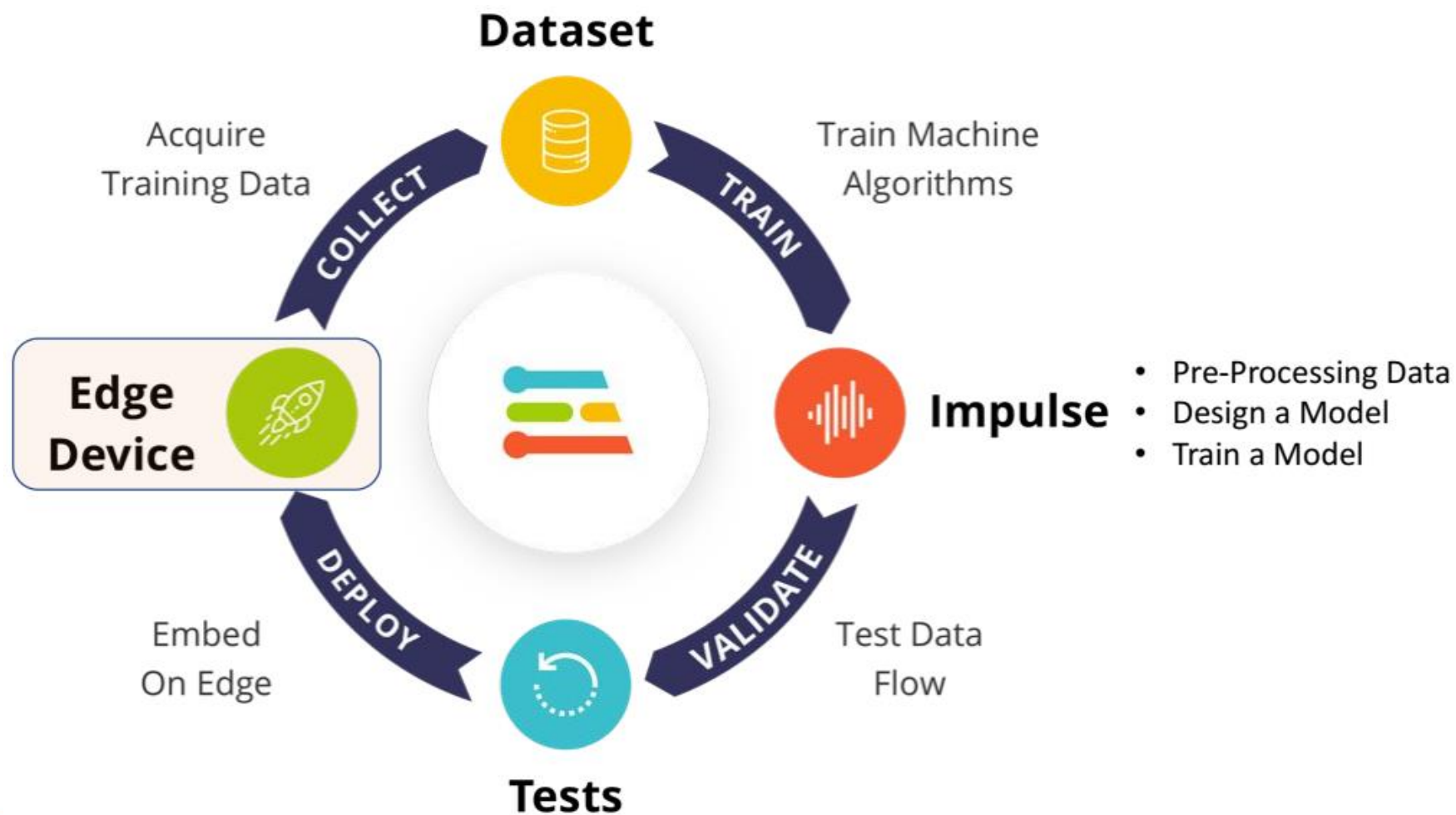
Classes

- Parado
- Sendo erguido
- Mar
- Caminhão

*Na sequência, projetaremos,
treinaremos e testaremos o
modelo.*

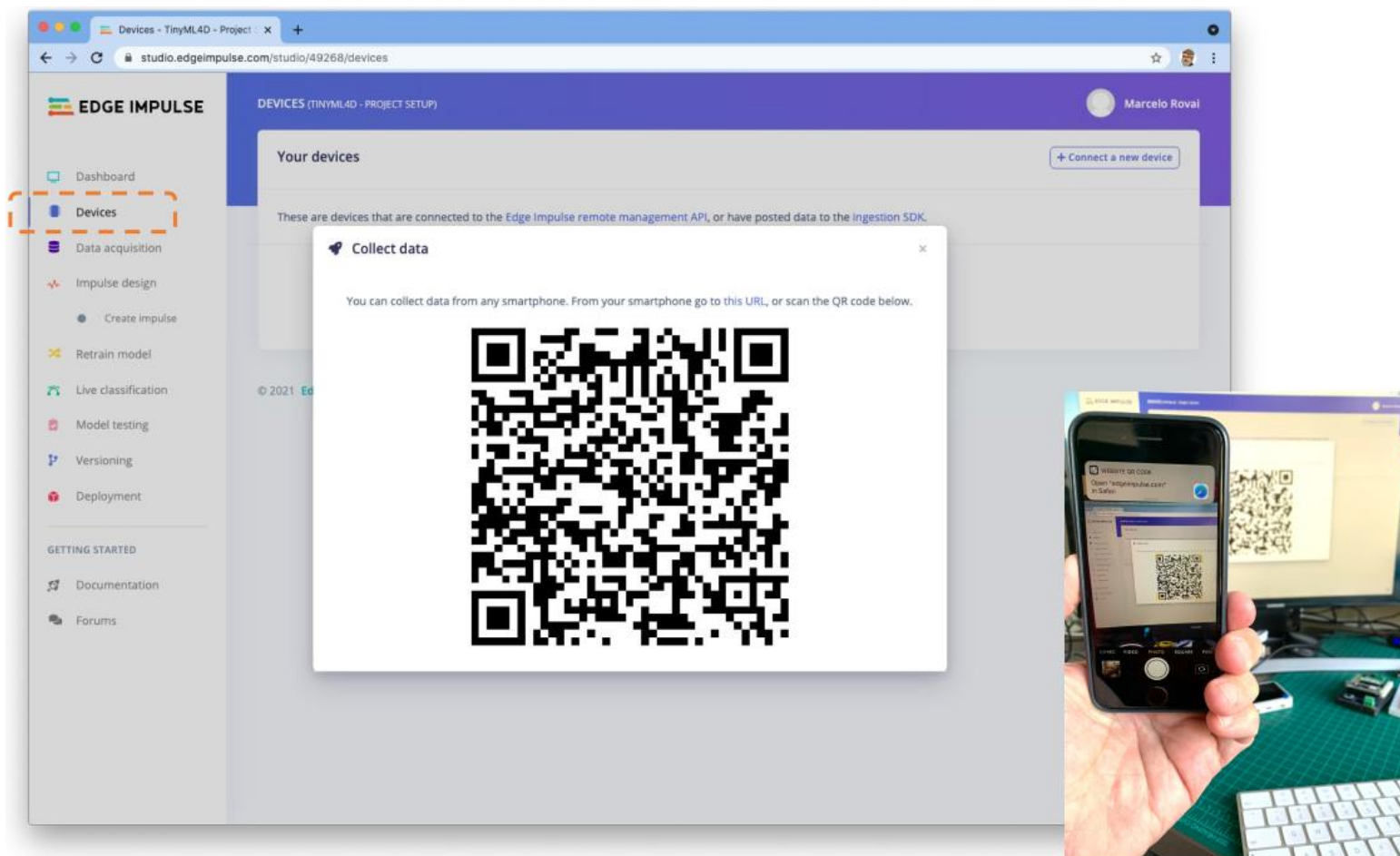
Motion Classification

Passos para criação do classificador via Edge Impulse



Motion Classification

1. Conectando o dispositivo para coleta dos dados



Motion Classification

2. Coleta dos dados

Collect Data

The screenshot displays the Edge Impulse Studio web interface. On the left, a sidebar contains navigation links: Dashboard, Devices, Data acquisition (highlighted with a blue arrow), Impulse design, Create impulse, Spectral Analysis, Neural Network (Ko...), EON Tuner, Retrain model, Live classification, Model testing, Versioning, Deployment, GETTING STARTED, Documentation, and Forums. The main area is titled 'DATA ACQUISITION (SCITINYML-MOTION-PROJECT)' and includes tabs for 'Training data' and 'Test data'. It shows 'DATA COLLECTED 5m 13s' and a 'TRAIN / TEST SPLIT 80% / 20%'. Below this is a table of 'Collected data' with columns for Sample Name, Label, Added, Length, and a status icon. The table lists 15 samples, with the last 5 labeled 'maritime'. On the right, the 'Record new data' section is highlighted with an orange dashed box. It includes a 'Device' dropdown (phone_kqfrazik), a 'Label' dropdown (maritime), a 'Sensor' dropdown (Accelerometer), a 'Sample length (ms.)' input (10000), and a 'Frequency' dropdown (62.5Hz). A 'Start sampling' button is also highlighted with an orange dashed box and a yellow arrow. Below the 'Record new data' section is a 'Sensor' dropdown menu showing 'Accelerometer' (selected), 'Microphone', and 'Camera'. On the far right, a smartphone mockup displays the 'Data collection' screen with a progress ring showing 4s.

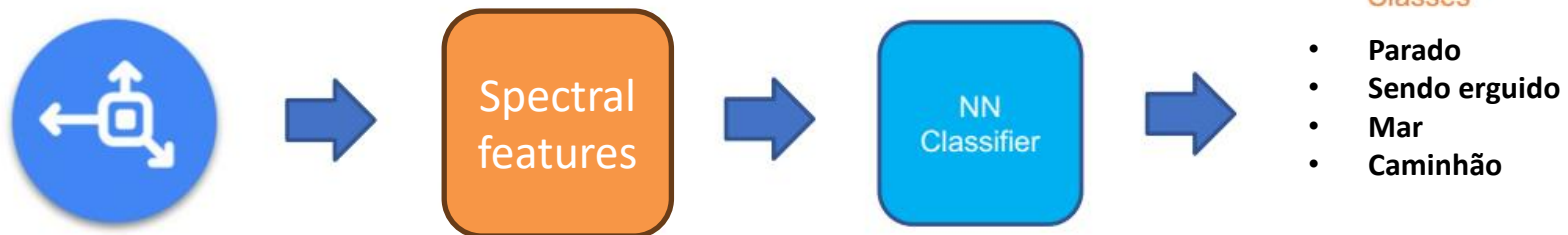
SAMPLE NAME	LABEL	ADDED	LENGTH
idle.2hstvpk2	idle	Oct 14 2021, 17:54:22	10s
idle.2hstuauf	idle	Oct 14 2021, 17:53:34	10s
idle.2hstt0q3	idle	Oct 14 2021, 17:53:16	10s
idle.2hstt9dk	idle	Oct 14 2021, 17:53:00	10s
idle.2hstsp4a	idle	Oct 14 2021, 17:52:43	10s
idle.2hstrkad	idle	Oct 14 2021, 17:52:06	10s
idle.2hstr3kf	idle	Oct 14 2021, 17:51:49	10s
idle.2hstqaj	idle	Oct 14 2021, 17:51:32	10s
maritime.2hstpk3	maritime	Oct 14 2021, 17:51:01	10s
maritime.2hsto9ki	maritime	Oct 14 2021, 17:50:16	10s
maritime.2hstnnqu	maritime	Oct 14 2021, 17:49:58	10s
maritime.2hatn60c	maritime	Oct 14 2021, 17:49:40	10s

Motion Classification

3. Criar um impulse

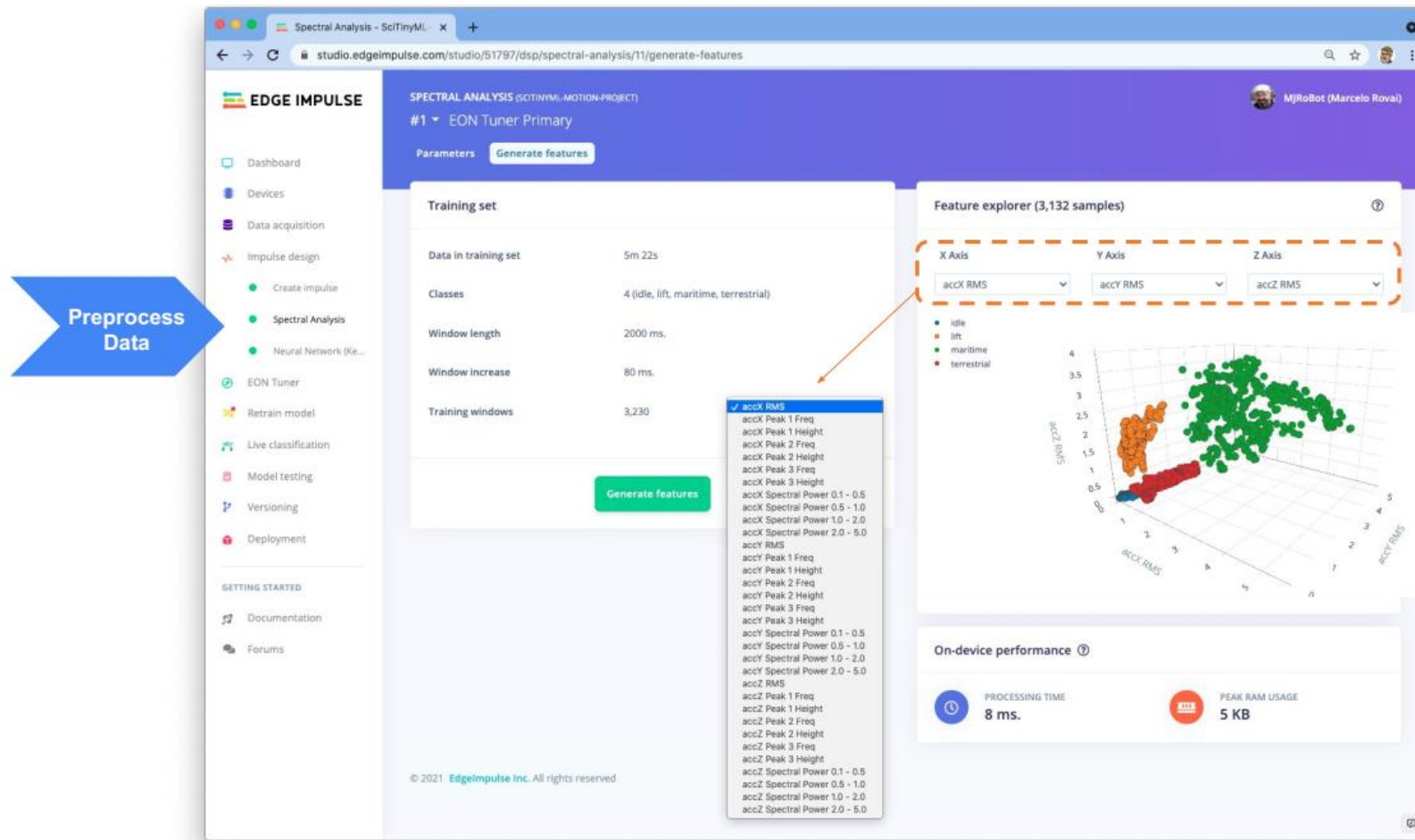
The screenshot shows a web interface for creating an impulse. It consists of four main panels:

- Time series data** (red panel):
 - Axes: accX, accY, accZ
 - Window size: 2000 ms.
 - Window increase: 80 ms.
 - Frequency (Hz): 62.5
 - Zero-pad data: ☒
- Spectral Analysis** (white panel):
 - Name: Spectral Analysis
 - Input axes: ☒ accX, ☒ accY, ☒ accZ
- Neural Network (Keras)** (purple panel):
 - Name: Neural Network (Keras)
 - Input features: ☒ Spectral Analysis
 - Output features: 4 (idle, lift, maritime, terrestrial)
- Output features** (teal panel):
 - 4 (idle, lift, maritime, terrestrial)
 - Save Impulse button



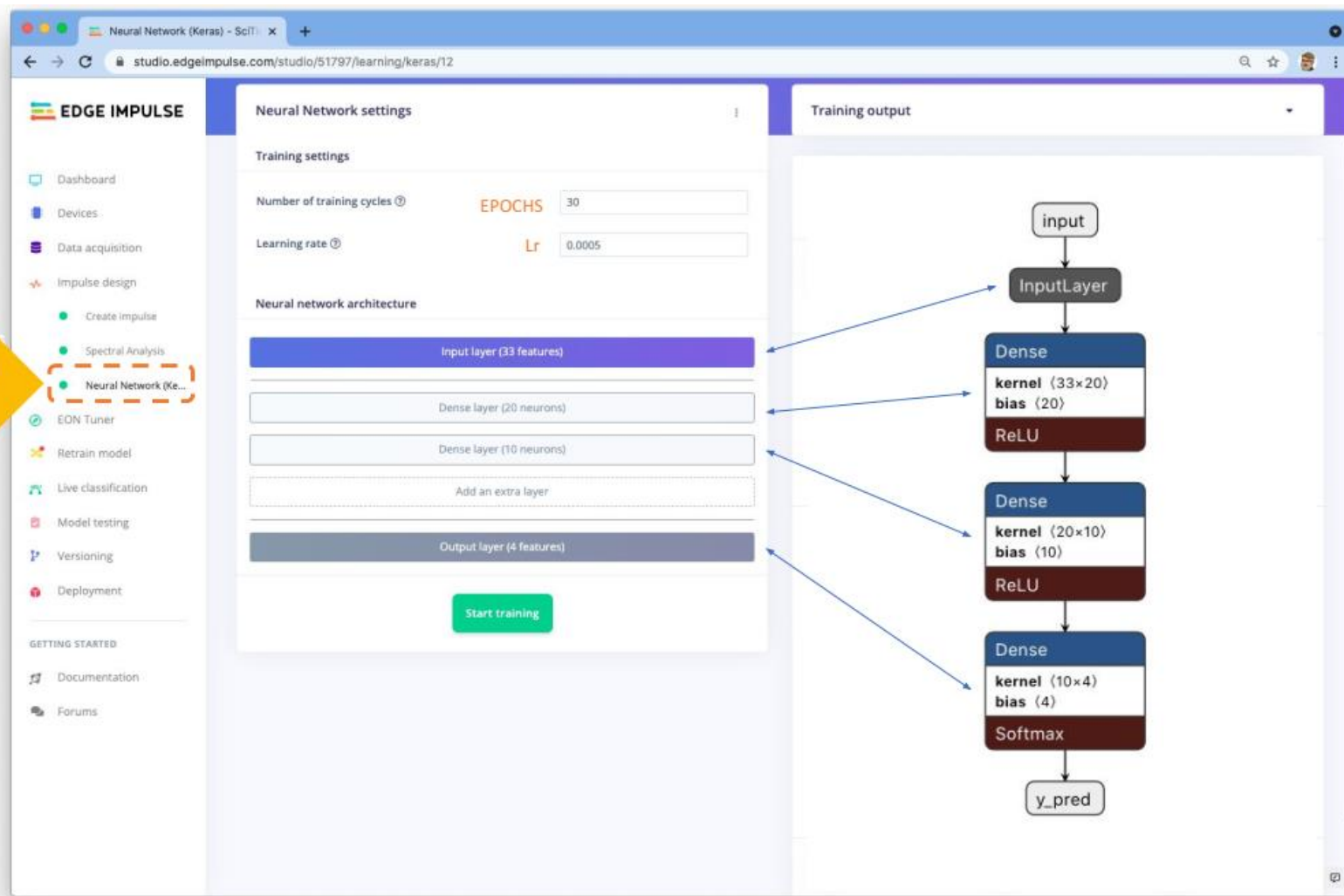
Motion Classification

4. Pré-processamento dos dados



Motion Classification

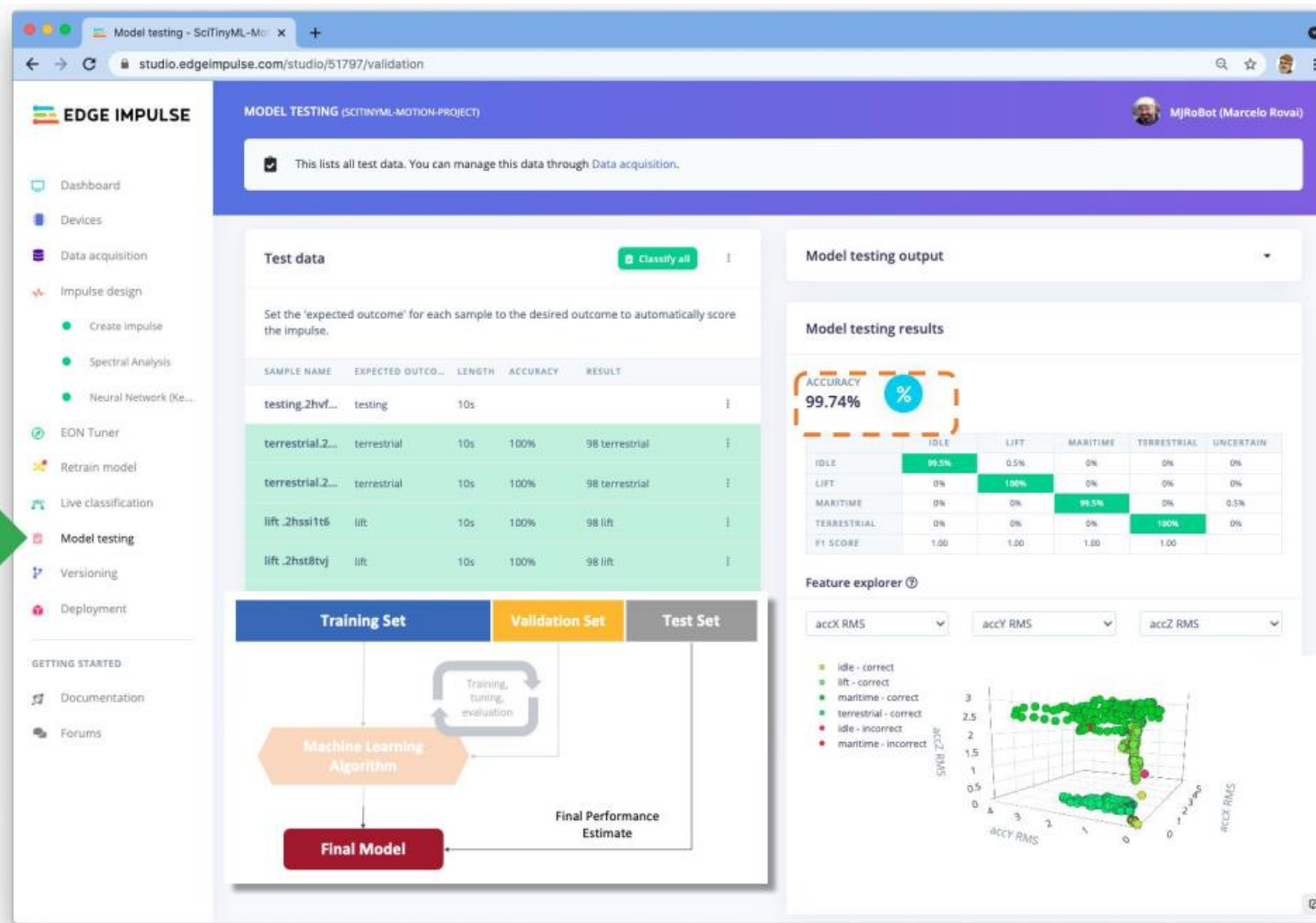
5. Projetar um modelo, treiná-lo e avaliar o resultado



Design a
Model

Motion Classification

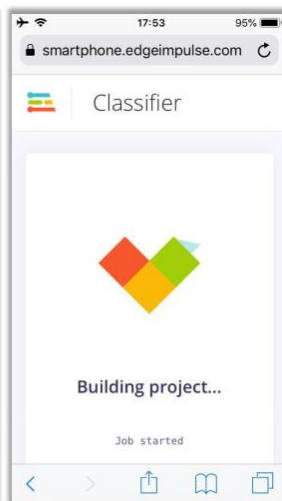
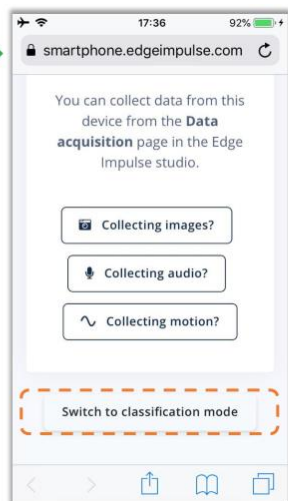
6. Avaliar a capacidade de generalização do modelo



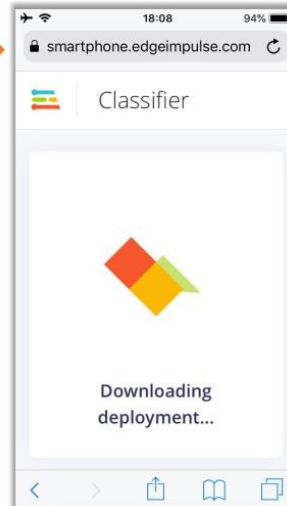
Motion Classification

7. Converter o modelo, fazer o deploy e realizar inferências

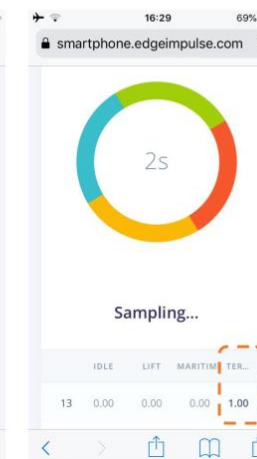
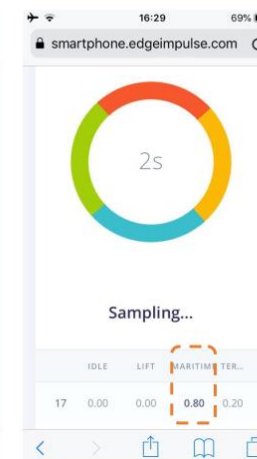
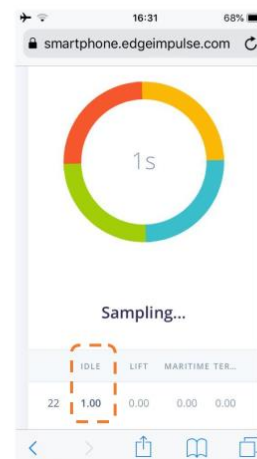
Convert Model



Deploy Model



Make Inferences



Motion Classification

8. Executando no Arduino nano-33 BLE sense

The screenshot shows the Edge Impulse Studio deployment page. The left sidebar contains navigation links: Dashboard, Devices, Data acquisition, Impulse design (with sub-links: Create impulse, Spectral features, NN Classifier), EON Tuner, Retrain model, Live classification, Model testing, Versioning, and Deployment (highlighted with a red arrow). Below these are links for GETTING STARTED, Documentation, and Forums. The main content area is titled 'DEPLOYMENT (TEST01 - NANO MOTION CLASSIFICATION)'. It has a 'Deploy your impulse' section with instructions. Below that is a 'Create library' section with a red box highlighting the 'Arduino library' option among others like C++ library, CubeMX CMSIS-PACK, WebAssembly, and TensorRT library. The 'Build firmware' section lists various development boards, including 'Arduino Nano 33 BLE Sense'. On the right, the 'Build output' panel shows the progress of the build, ending with 'Job completed'.

EDGE IMPULSE

DEPLOYMENT (TEST01 - NANO MOTION CLASSIFICATION)

Deploy your impulse

You can deploy your impulse to any device. This makes the model run without an internet connection, minimizes latency, and runs with minimal power consumption. [Read more.](#)

Create library

Turn your impulse into optimized source code that you can run on any device.

C++ library

Arduino library

CubeMX CMSIS-PACK

WebAssembly

TensorRT library

Build firmware

Or get a ready-to-go binary for your development board that includes your impulse.

ST IoT Discovery Kit

Arduino Nano 33 BLE Sense

Eta Compute ECM3532 AI Sensor

Silabs Thunderboard Sense 2

Himax WE-I Plus

Nordic nRF52840 DK + IKS02A1

Nordic nRF5340 DK + IKS02A1

Nordic nRF9160 DK + IKS02A1

Nordic Thingy 91

Build output

Creating job... OK (ID: 1646786)

Writing templates...

Writing templates OK

Copying Edge Impulse SDK...

Copying Edge Impulse SDK OK

Compiling EON model...

Compiling EON model OK

Removing clutter and updating headers...

Removing clutter and updating headers OK

Creating archive...

Job started

Creating archive OK

Job completed

Motion Classification

8. Executando no Arduino nano-33 BLE sense

The screenshot displays the Edge Impulse IDE interface. On the left is a sidebar with navigation options: Dashboard, Devices, Data acquisition, Experiments, EON Tuner, and Impulse design. The main area shows 'MODEL OPTIMIZATIONS' with a section for 'EON™ Compiler' and options for 'Quantized (int8)' and 'Unoptimized (float32)'. A modal dialog box titled 'Built Arduino library' is centered on the screen, featuring a green checkmark icon. The dialog contains the following text: 'Add this library through the Arduino IDE via: Sketch > Include Library > Add .ZIP Library...'. Below this, it states 'Examples can then be found under: File > Examples > anomaly_inferencing'. The background interface is dimmed, showing a 'Build' button at the bottom.

EDGE IMPULSE

Dashboard
Devices
Data acquisition
Experiments
EON Tuner
Impulse design

Create Impulse
Spectral features
Classifier
Anomaly detection
Retrain model
Live classification
Model testing

Upgrade Plan

MODEL OPTIMIZATIONS
Model optimizations can increase on-device performance but may reduce accuracy.

EON™ Compiler
Same accuracy, 54% less RAM, 60% less ROM.

Quantized (int8)
Selected ✓

Unoptimized (float32)
Select

To compare model accuracy, run

Estimate for Arduino Nano 33 BLE Sense (Cortex-M4F 64MHz) [Change target](#)

Built Arduino library

Add this library through the Arduino IDE via:
Sketch > Include Library > Add .ZIP Library...

Examples can then be found under:
File > Examples > anomaly_inferencing

Build

Job scheduled at 22 May 2025 14:35:39
Job started at 22 May 2025 14:35:42
Writing templates...
Writing templates OK

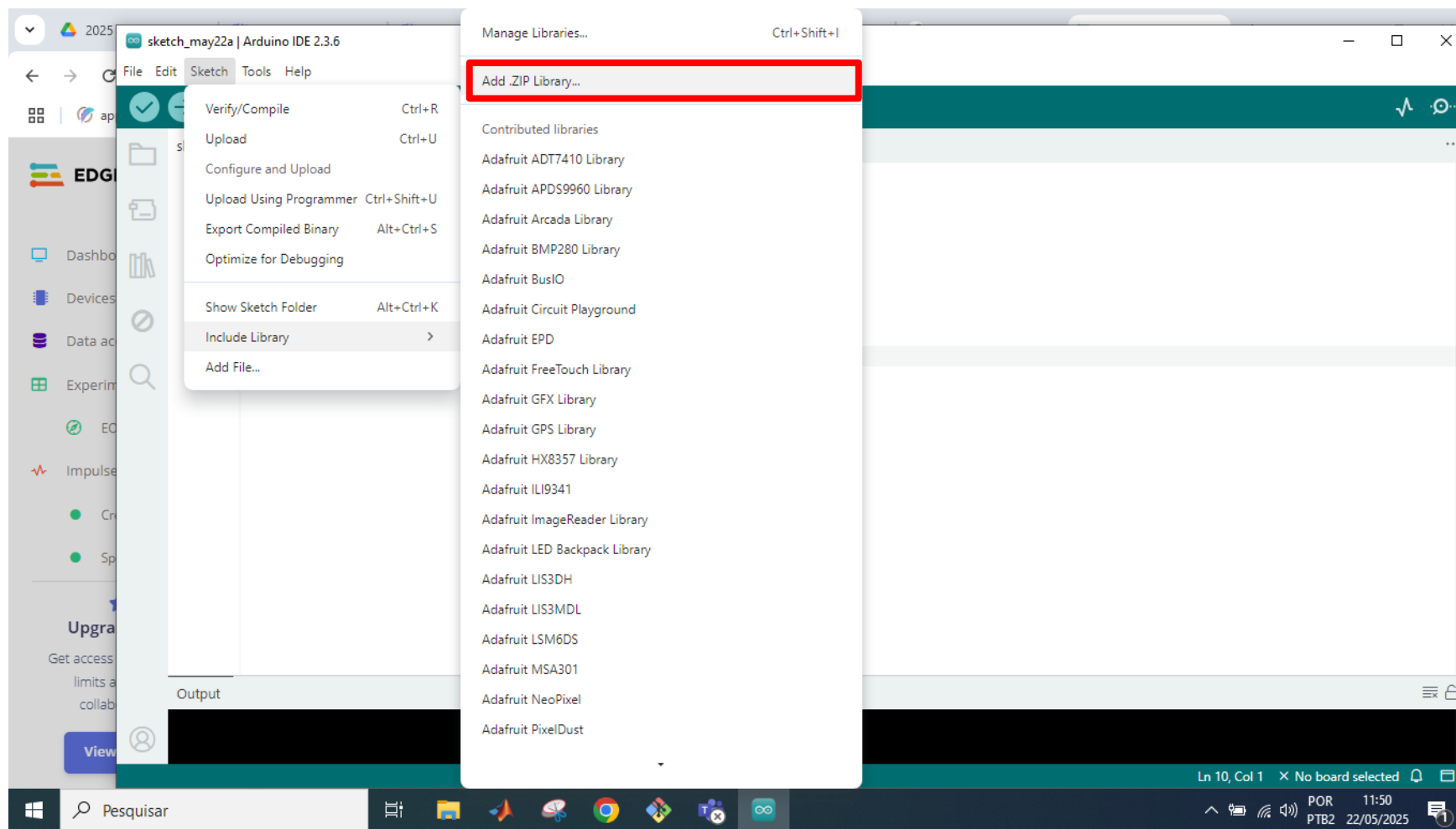
Job scheduled at 22 May 2025 14:35:52
Job started at 22 May 2025 14:35:53
Copying Edge Impulse SDK...
Copying Edge Impulse SDK OK

Compiling EON model...
Compiling EON model OK

Removing clutter and updating headers...
Removing clutter and updating headers OK

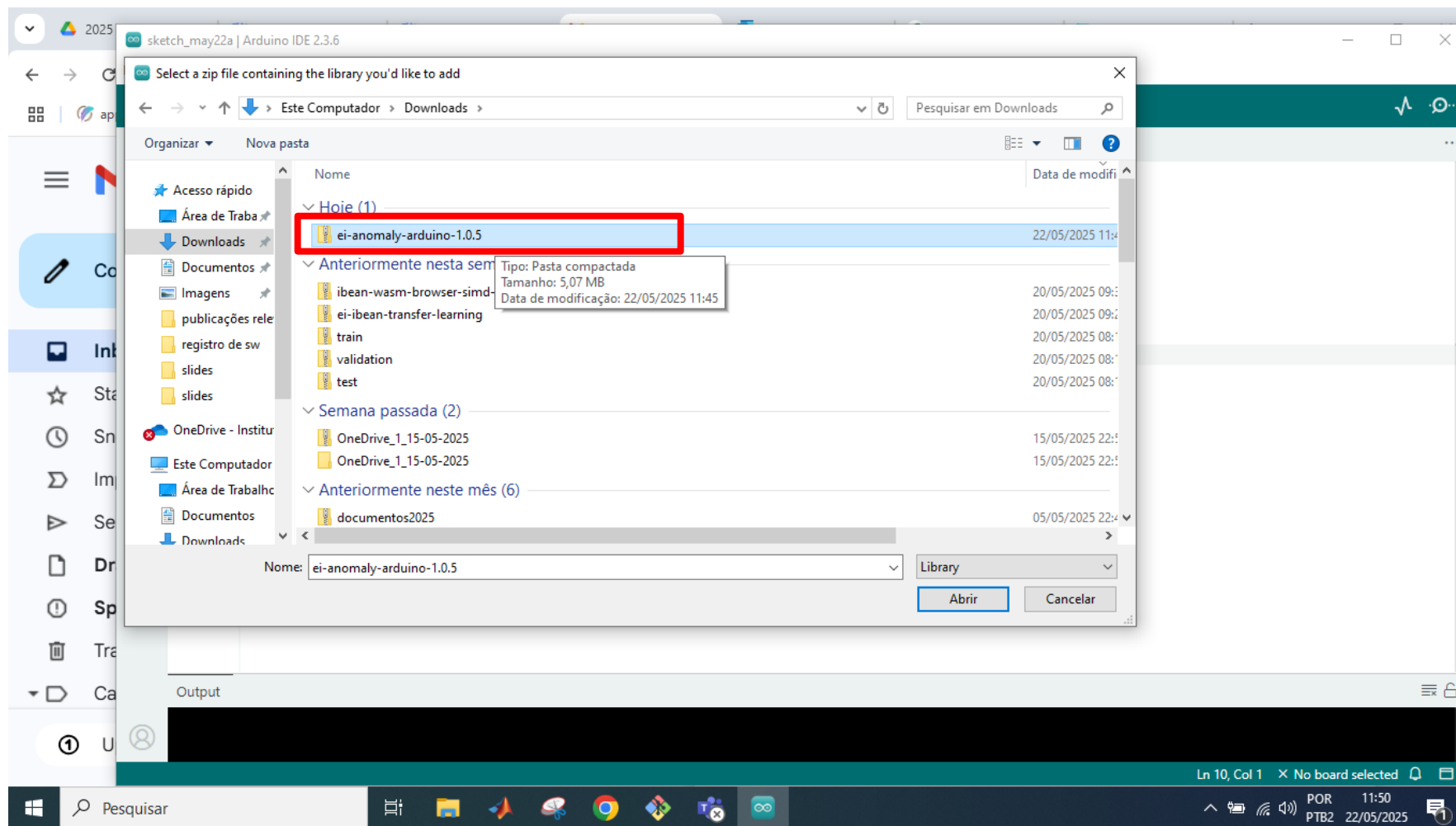
Motion Classification

8. Executando no Arduino nano-33 BLE sense



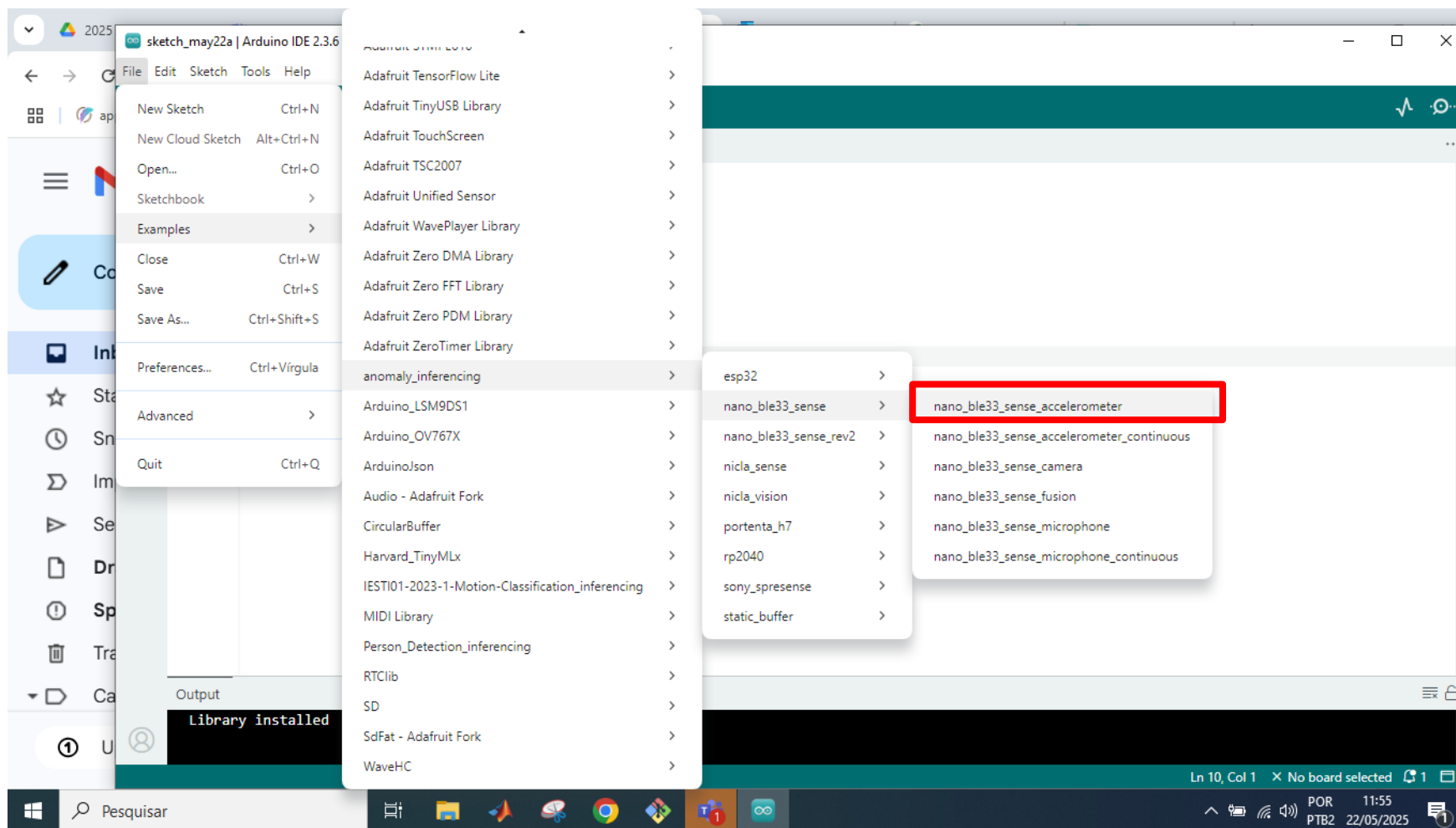
Motion Classification

8. Executando no Arduino nano-33 BLE sense



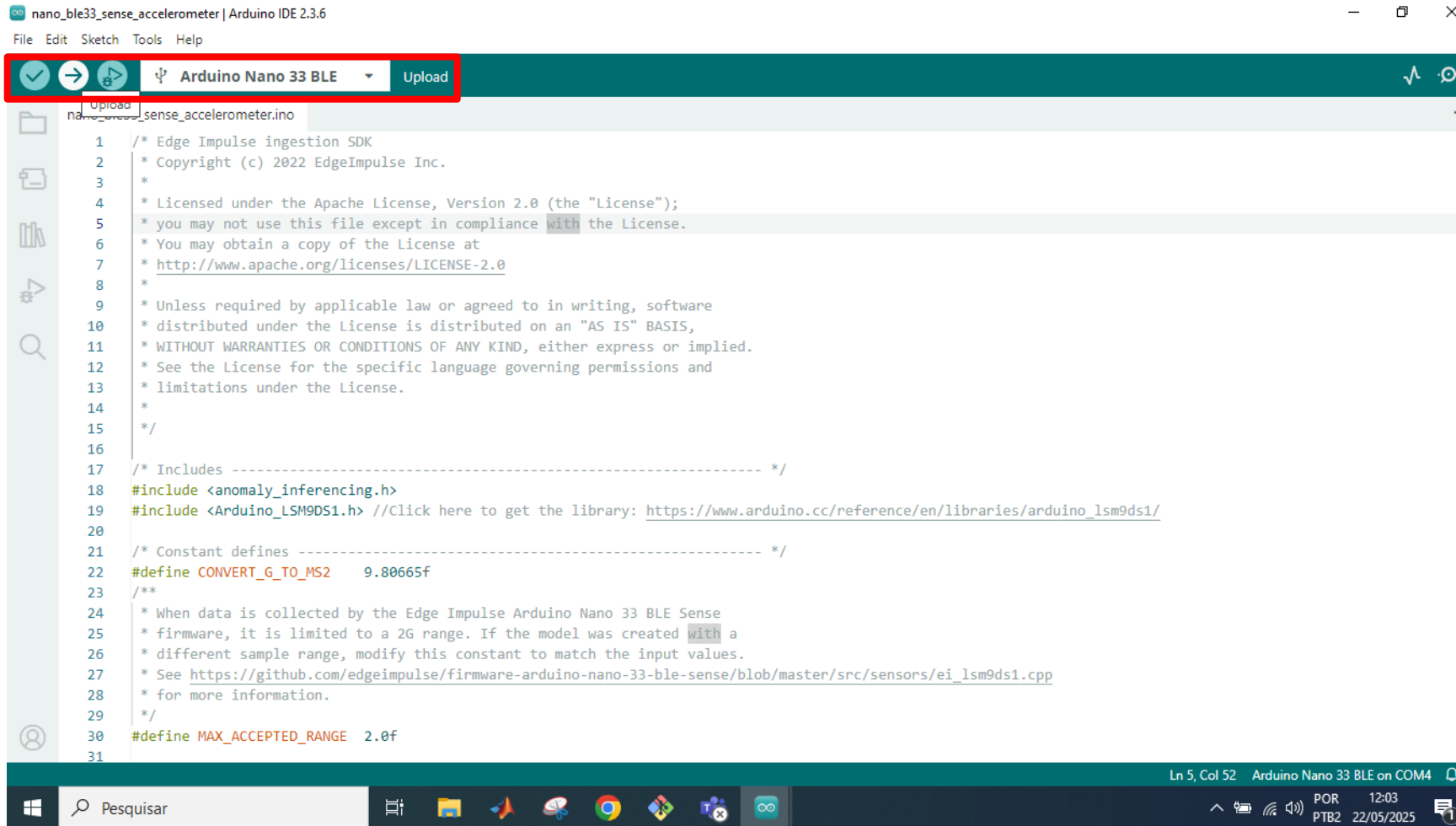
Motion Classification

8. Executando no Arduino nano-33 BLE sense



Motion Classification

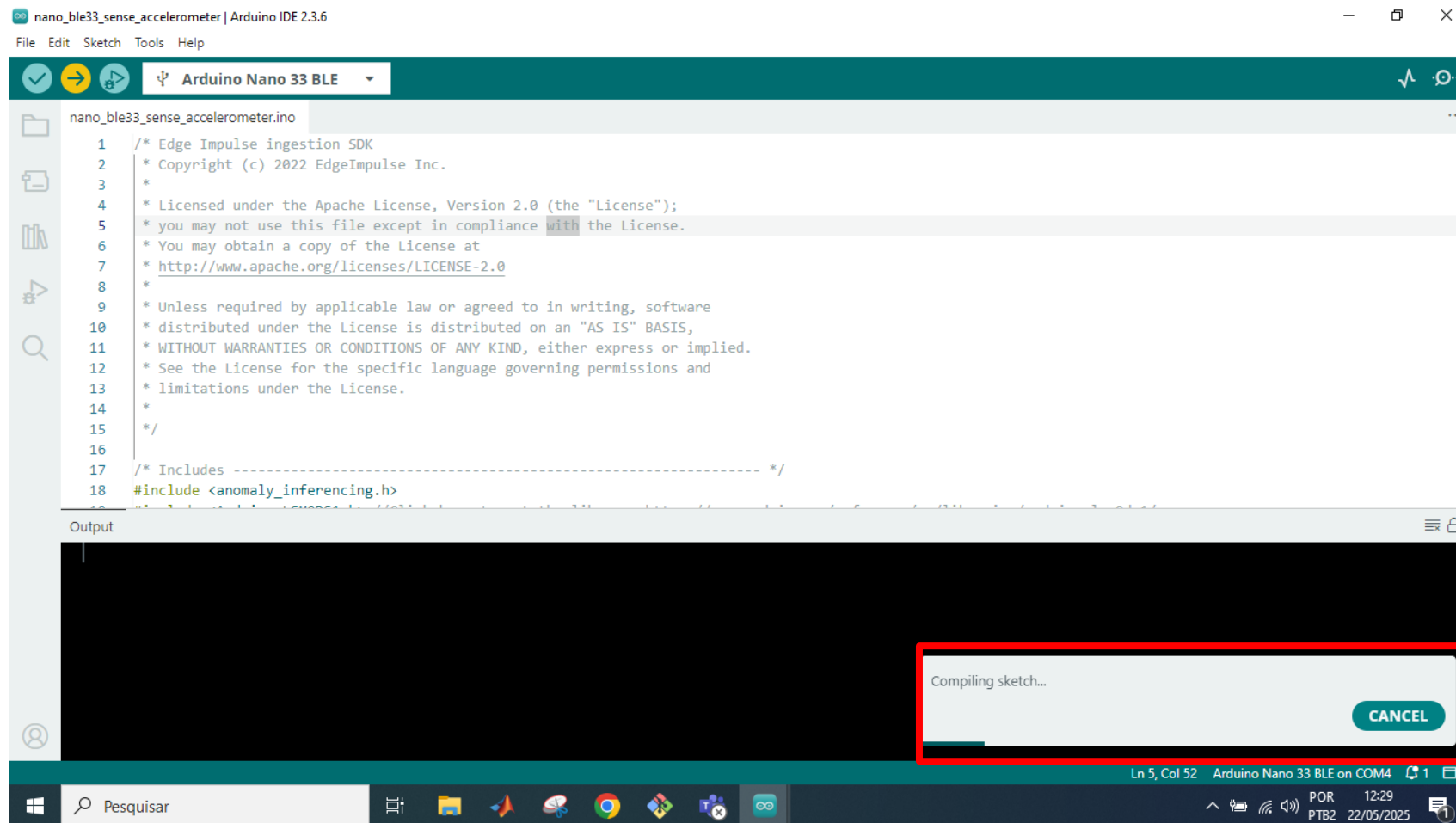
8. Executando no Arduino nano-33 BLE sense



```
1  /* Edge Impulse ingestion SDK
2  * Copyright (c) 2022 EdgeImpulse Inc.
3  *
4  * Licensed under the Apache License, Version 2.0 (the "License");
5  * you may not use this file except in compliance with the License.
6  * You may obtain a copy of the License at
7  * http://www.apache.org/licenses/LICENSE-2.0
8  *
9  * Unless required by applicable law or agreed to in writing, software
10 * distributed under the License is distributed on an "AS IS" BASIS,
11 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
12 * See the license for the specific language governing permissions and
13 * limitations under the License.
14 *
15 */
16
17 /* Includes ----- */
18 #include <anomaly_inferencing.h>
19 #include <Arduino_LSM9DS1.h> //Click here to get the library: https://www.arduino.cc/reference/en/libraries/arduino\_lsm9ds1/
20
21 /* Constant defines ----- */
22 #define CONVERT_G_TO_MS2 9.80665f
23 /**
24 * When data is collected by the Edge Impulse Arduino Nano 33 BLE Sense
25 * firmware, it is limited to a 2G range. If the model was created with a
26 * different sample range, modify this constant to match the input values.
27 * See https://github.com/edgeimpulse/firmware-arduino-nano-33-ble-sense/blob/master/src/sensors/ei\_lsm9ds1.cpp
28 * for more information.
29 */
30 #define MAX_ACCEPTED_RANGE 2.0f
31
```

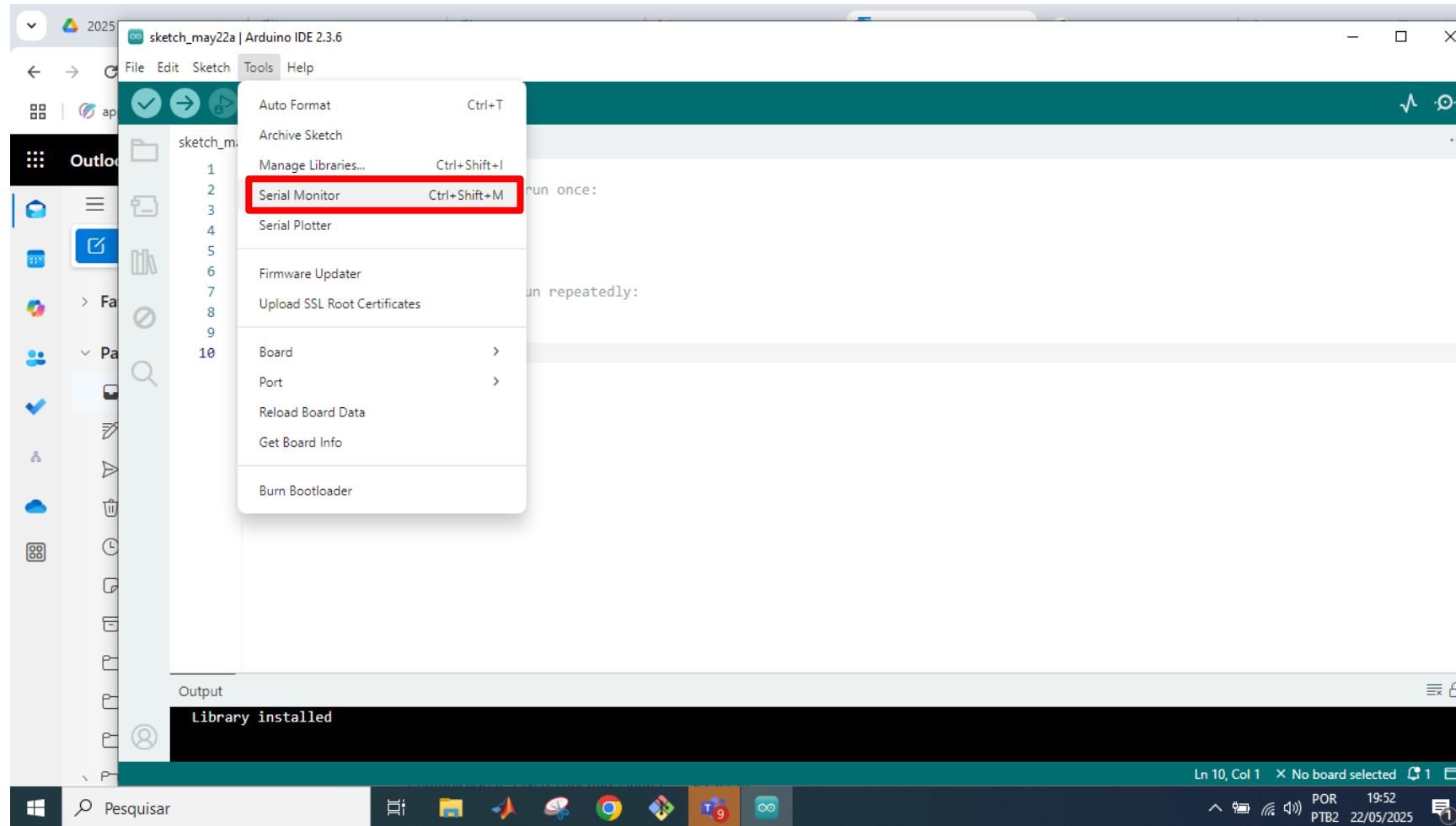
Motion Classification

8. Executando no Arduino nano-33 BLE sense



Motion Classification

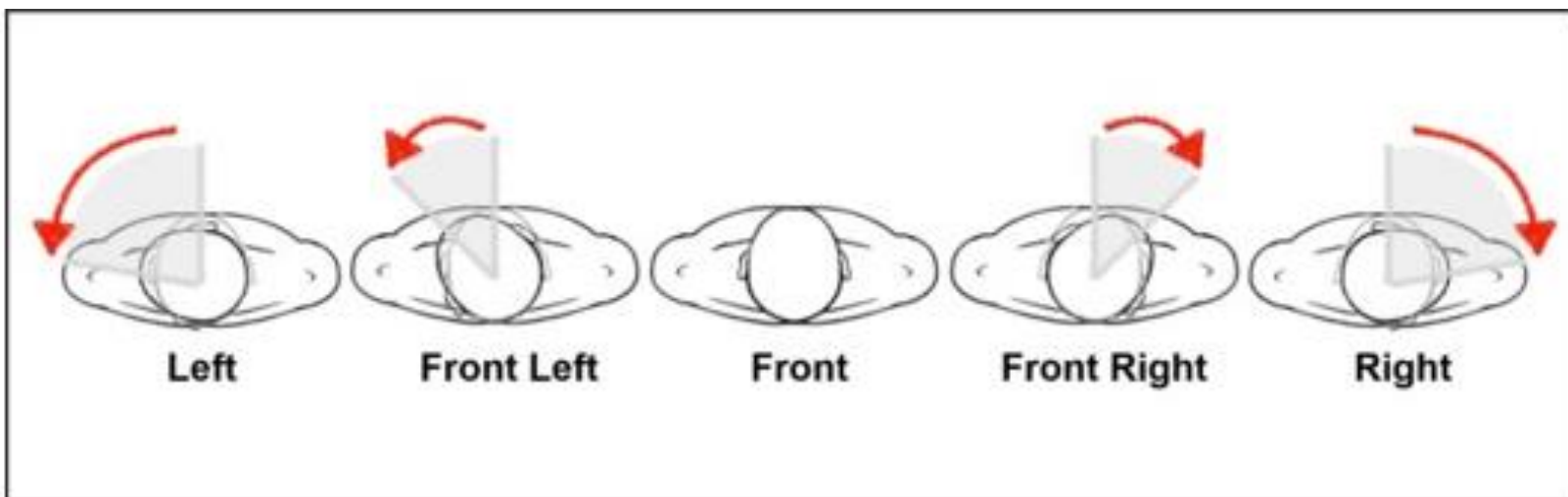
8. Executando no Arduino nano-33 BLE sense



Edge Impulse

Trabalho: Movendo uma cadeira de rodas com a cabeça

- Criar um dataset para classificação de movimento para movimentação de uma cadeira de rodas com uso do movimento da cabeça, treinar pelo menos três movimentos no Edge Impulse.
- Exemplo:



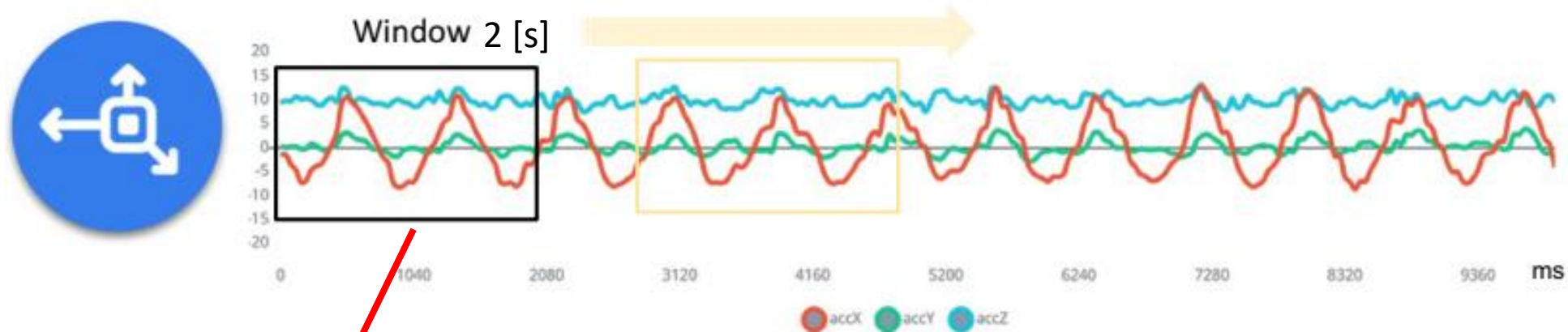
Spectral Analysis

Referências

- [1] https://github.com/Mjrovai/UNIFEI-UESTI01-TinyML/blob/main/00_Curse_Folder/2_Applications_Deploy/Class_20/EI_Studio-Spectral_Analysis-Updated_March-23.pdf
- [2] https://colab.research.google.com/github/Mjrovai/TinyML4D/blob/main/SciTinyM-2023/Edge_Impulse-Spectral_Analysis_Block/Edge_Impulse_Spectral_Analysis_Block_V3.ipynb#scrollTo=09cf0268

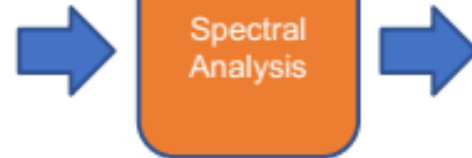
Anexo I

Motion Classification



375 Raw Features

- Raw Data from sensor

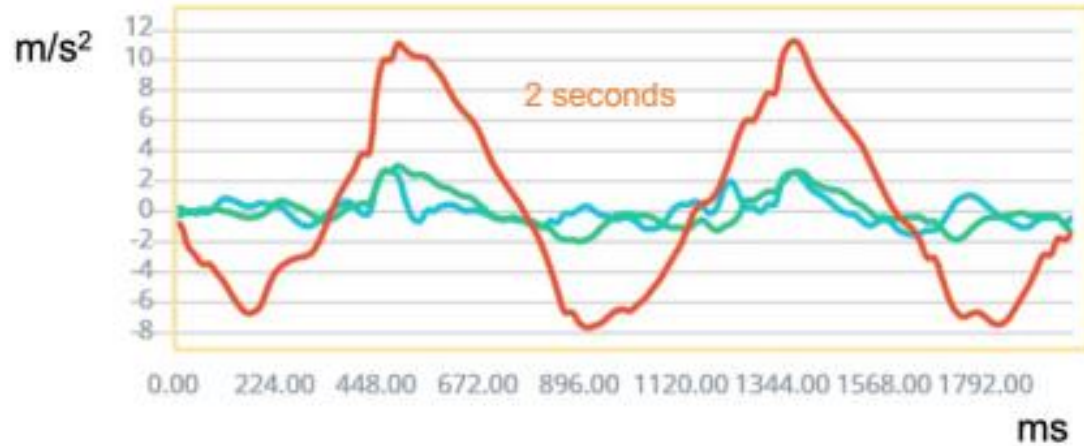


*Etapa do pré-processamento
usando o bloco “spectral
analysis”*

* O bloco foi substituído pelo “Spectral features” na versão mais recente

<https://docs.edgeimpulse.com/docs/edge-impulse-studio/processing-blocks/spectral-features>

Motion Classification



Manual Feature Extraction



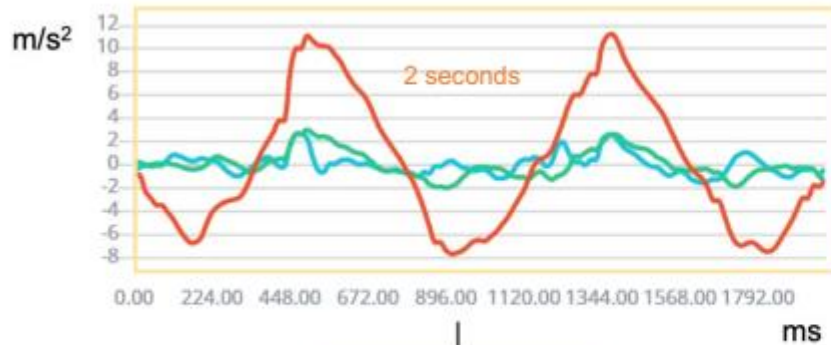
3 RMS (Root Mean Square) values
- one for each axis (x, y, z)

$$x_{\text{RMS}} = \sqrt{\frac{1}{n} (x_1^2 + x_2^2 + \dots + x_n^2)}.$$

125

1. Etapa do pré-processamento:
Calcular o valor RMS de cada um dos eixos.

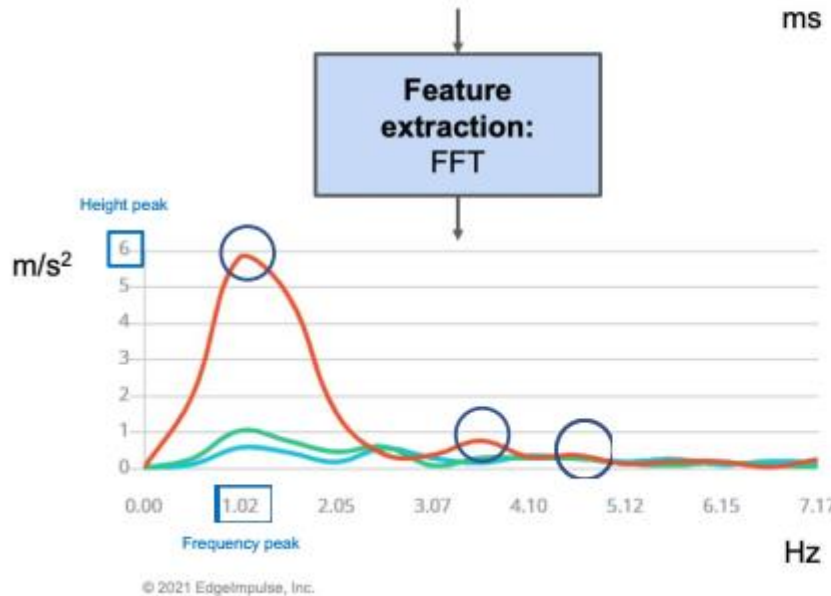
Motion Classification



Manual Feature Extraction

➔ 3 RMS

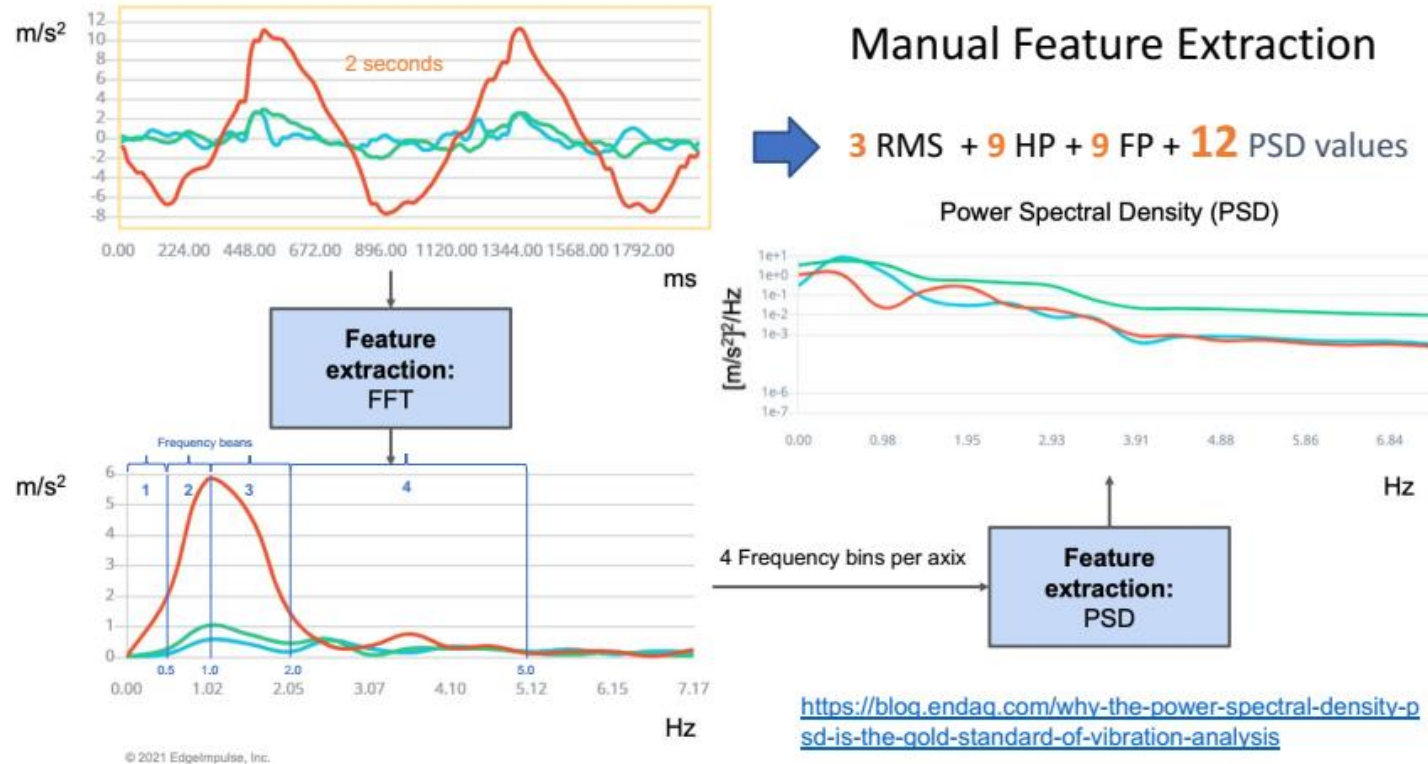
2. Etapa do pré-processamento:
Calcular a FFT dos sinais do 3 eixos.
Usamos como atributos de entrada:
3 valores de pico e suas respectivas
frequências para cada eixo.
Totalizando 18 atributos.



➔ 9 Height + 9 Freq. peak values

Extrair as frequências dos sinais

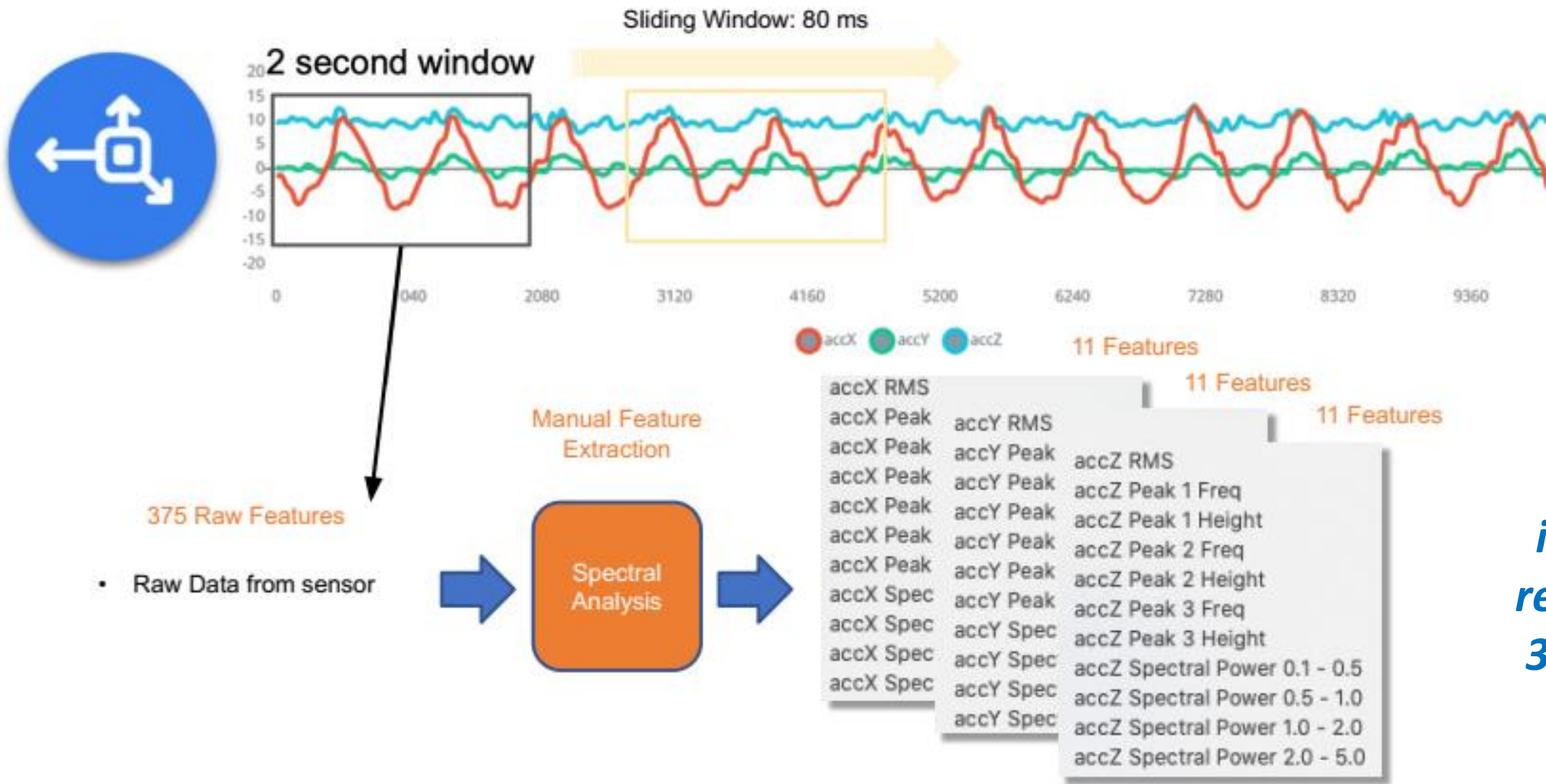
Motion Classification



3. Etapa do pré-processamento:
Densidade espectral de potência.

Método de Welch: Divide-se a largura de banda em 4 e calcula-se a densidade de potência para cada região. Assim, temos 4 regiões * 3 eixos = 12 atributos.

Motion Classification



Portanto, ao invés de 375 a rede neural terá 33 atributos de entrada.

Motion Classification



A janela de 2 [s] desliza de 80 [ms].

